

The I2S-anti family: three ways input-to-state copying backfires

Backdrop. *I2S-anti* (I2S-A) collects the roadblocks where input-to-state copying *hurts*: a non-I2S arm resolves the branch but the I2S-bearing arm blocks it. It is the *same* operand substitution that powers I2S-pro, turned against the fuzzer — `cmplog` logs a comparison operand and `I2SRandReplace` splices it back in, but here the spliced value steers the corpus *away* from what the branch needs. Every case shares one measured signature: on the campaign corpus the I2S arm is *depleted* of the byte the winning arm reaches, so the same `operand_enrichment` tool fires with the sign flipped — `signed_target_enrich` < -0.3 (versus $\geq +1.0$ for I2S-pro). The three categories differ only in *what decoy* the substitution injects. The backfire shows up against either baseline — *naive* $\succ$ *cmplog* or *value_profile* $\succ$ *vpc* (the same substitution hurting plain havoc, or hurting on top of the value-profile gradient); we merge both because the program mechanism is identical. Counts are validated roadblocks (n).

A note on the metric. `signed_target_enrich` is a sign-preserving, max-magnitude *per-offset* $\log_2$ frequency ratio (`cmplog` vs *naive* at the gate offset, with an $\epsilon=10^{-6}$ floor). Its *sign* is the classifier signal (negative = the I2S arm is depleted of the target byte); its magnitude saturates when the byte is absent at an offset, so a value like -13.6 just means “essentially gone,” not a precise ratio.

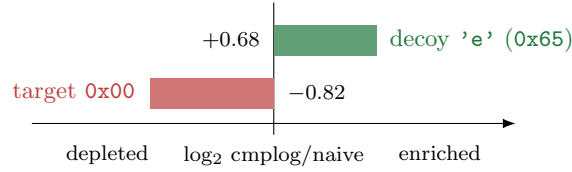
1. A rival operand wins the harvest i2s_anti_target_depletion ($n = 25$)

The gate needs a specific byte value, and *naive* havoc lands it by chance — but I2S keeps splicing a *different*, abundant operand it harvested from some *other* comparison in the program into the same position, pinning the `cmplog` corpus onto that decoy and starving it of the value the branch wants.

```
// libpng pngutil.c : png_handle_cHRM()      (cHRM chunk, length 32)
xy.blue_x = png_get_fixed_point(buf + 24);    // parse the blue_x sub-field
if (xy.blue_x == PNG_FIXED_ERROR || ...)      // <-- blocker (libpng/3977)
    png_chunk_benign_error(...);              // taken when blue_x is out-of-
range
```

The branch is taken when `blue_x` parses to an out-of-range fixed-point (`PNG_FIXED_ERROR`). *naive*’s random bytes hit that easily (target byte `0x00`); but `cmplog` harvests the ASCII float exponent ‘e’ (`0x65`) from *other* numeric comparisons in the parser (e.g. from “1.0e0”, “65.535”) and splices it into the `blue_x` bytes, producing an in-range value that does *not* trip the gate — so the I2S corpus is pinned to the decoy and depleted of the target.

Tool: <code>operand_enrichment</code>
Metric: <code>signed_target_enrich</code> (target byte) and <code>decoy_enrich</code> (the rival byte), both $\log_2$ <code>cmplog</code> / <i>naive</i> ratios
Verify rule (deterministic arbiter): <code>skip != no_gate_signature AND signed_target_enrich < -0.3</code>
Real example: <code>libpng/3977</code> (cHRM handler, <code>pngutil.c:1283</code>) — the decoy ‘e’ fills 63% of the <code>cmplog</code> corpus (vs 39% <i>naive</i>) while the target byte drops to 8.9% (vs 15.7% <i>naive</i>): <code>signed_target_enrich</code> = -0.82 , <code>decoy_enrich</code> = $+0.68$.



I2S steers the cmplog corpus toward the wrong (decoy) operand and away from the value the branch needs.

2. Blind substitution clobbers assembled structure `i2s_anti_structural_byte_corruption` ($n = 10$)

Here there is *no* rival operand. *naive* havoc *assembles* the correct structural bytes, and `I2SRandReplace` blindly overwrites that exact position with a logged value — pure destruction, not redirection. The clean fingerprint is shape `LWWW`: cmplog is the *sole* loser (naive, value_profile, and vpc all resolve).

```
// harfbuzz hb-ot-shaper-indic.cc
// : initial_reordering_consonant_syllable()
if (start + 3 <= end && // <-- blocker (harfbuzz/5831): syllable >= 3
    glyphs
    ...)                // needs a long repeated glyph run
```

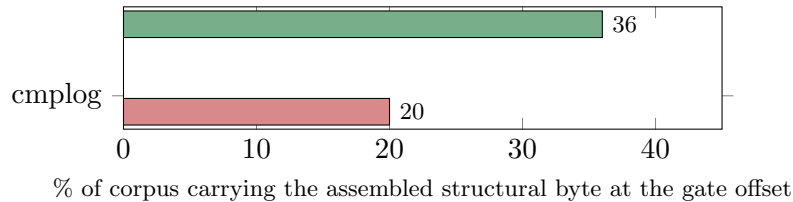
The gate needs a Kannada syllable spanning ≥ 3 glyphs, which naive builds from long repeated multi-byte glyph runs. I2S overwrites bytes inside those runs with harvested constants, shortening the run below the threshold, so the cmplog corpus is depleted of the assembled structure.

Tool: `operand_enrichment`

Metric: `signed_target_enrich`; the decoy fractions are small and *diffuse* (no single dominant rival)

Verify rule (deterministic arbiter): `signed_target_enrich < -0.3`

Real example: `harfbuzz/5831` (`initial_reordering_consonant_syllable:479`) — the structural byte is present in 36% of naive seeds but only 20% of cmplog’s; `signed_target_enrich` = -13.6 (saturated = essentially erased at the operand offset), with only diffuse displacement (decoy fractions 3% vs 0.8%).



naive assembles the structural byte; I2S overwrites it, so cmplog carries it far less often.

3. A validity trap starves a semantic gate `i2s_anti_decoy_overfit` ($n = 14$)

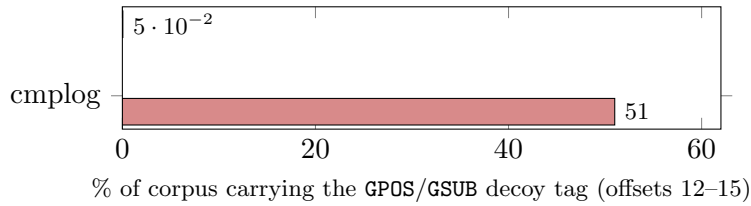
The most subtle case. The decisive condition is *semantic* — not a byte at an offset — and I2S keeps re-asserting harvested *structural* literals that hold the input on a “valid” path, starving the branch of the semantic input it needs.

```
// harfbuzz hb-common.cc : hb_script_get_horizontal_direction()
switch (script) {
...
case HB_SCRIPT_THAANA: // <-- blocker (harfbuzz/5323)
    return HB_DIRECTION_RTL; // needs Thaana-script text (U+0780..07BF)
```

}

The branch fires only when the decoded text is Thaana script. *naive* produces short, low-structure blobs that happen to decode to exotic scripts; but I2S keeps splicing the harvested OpenType table tags GPOS/GSUB into offsets 12–15, keeping the font structurally valid and ASCII/Latin — so it *reaches* the **switch** every trial but never *produces* the script the gate wants.

Tool: `operand_enrichment`
Metric: `signed_target_enrich / decoy_enrich`. The byte decoy is a structural *correlate* of the semantic gate, not the causal operand (a documented limit of a byte test on a semantic condition).
Verify rule (deterministic arbiter): `signed_target_enrich < -0.3`
Real example: `harfbuzz/5323` (`hb_script_get_horizontal_direction:594`, case `HB_SCRIPT_THAANA`) — the GPOS/GSUB decoy fills 51% of the `cmplg` corpus vs 0.05% of *naive*’s; `signed_target_enrich = -2.79`, `decoy_enrich = +11.3`. The label still fires for the right reason: `cmplg` is depleted of what resolves the branch.



I2S floods the corpus with valid table tags, keeping the font Latin and starving the Thaana-script gate.

At a glance

Category	<i>n</i>	Example	What the decoy is
target_depletion	25	libpng/3977	one abundant <i>rival</i> operand ('e')
structural_byte_corruption	10	harfbuzz/5831	none — diffuse destruction of assembled bytes
decoy_overfit	14	harfbuzz/5323	a structural <i>correlate</i> of a semantic gate (GPOS/GSUB)
I2S-A total	49		all share <code>signed_target_enrich < -0.3</code>

All counts and metric values are from `bench/dataset.jsonl` and the per-branch analysis cards; the verify rules are the deterministic arbiter rules (no LLM in the decision). `libpng/3977` is verified against local source; the `harfbuzz` gates are from their analysis cards (their corpora live on the other server).